

AIFP Repository Architecture

Document: AIFP-DOC-15 · Version: 1.0.0 · Governed by: AIFP-1 (Doc 01)

The official GitHub organization layout for the AiFinPay Paywall Protocol (AIFP).

Every repo follows the naming convention in Doc 13 and is published under

Apache-2.0. Org: github.com/aifinpay.

1. Organization Overview



2. Repository Catalog

Protocol & Specs

Repo	Contents	Source doc
openapi	openapi.yaml (3.1), lint config, codegen pipelines.	08
schemas	JSON Schema 2020-12 files + bundle; CI validation.	10

Repo	Contents	Source doc
conformance	Test vectors + runner (@aifinpay/conformance).	14

Backend

Repo	Contents
------	----------

Merchant SDK & Agent SDK

Repo	Package
agent-js	@aifinpay/agent
aifinpay-python	aifinpay-merchant , aifinpay-agent
aifp-go	github.com/aifinpay/aifp-go
aifinpay-rust	crate aifinpay
aifinpay-java	io.aifinpay:aifp
aifinpay-php	aifinpay/aifp
aifinpay-dotnet	AiFinPay

All SDK repos mirror the SDK Reference (Doc 11) and are generated/validated against `openapi` + `schemas` in CI.

Examples · Documentation · RFC · Website · Developer Portal

Repo	Contents	Doc
docs	Markdown content for the portal (Docs 01–15 rendered).	12
portal	Static-site generator + API Explorer + Receipt Inspector.	12
aifp (RFC section)	RFC + AIP process lives here.	01, 06
brand	Logos, color tokens, fonts, usage rules.	13
postman	Collection + sandbox/prod environments.	09

CI/CD

A central `.github` repo holds **reusable workflows** consumed by every repo: `lint` , `test` , `conformance` , `openapi-validate` , `schema-validate` , `codegen` , `release` (SemVer + changelog), `docs-preview` .

3. Standard Repo Layout (per SDK)

```

$ sdk-repo>
— src/                # library source
— tests/              # unit + conformance harness

```

```

— examples/           # language-specific runnable samples
— README.md           # install · quickstart · links to Doc 13
— CHANGELOG.md        # SemVer (Doc 06 §5)
— LICENSE             # Apache-2.0
— CONTRIBUTING.md      # → org contribution guide
— CODE_OF_CONDUCT.md
— SECURITY.md          # report channel → Security Council (Doc 14 §4)
— .github/
  — workflows/ci.yml   # uses reusable workflows from .github repo
  — ISSUE_TEMPLATE/
  — PULL_REQUEST_TEMPLATE.md

```

4. Issue Templates (org-level, in `.github`)

- **Bug report** — repro, expected vs actual, SDK/version, protocol version, request_id.
- **Feature request** — problem, proposed solution, alternatives. Large changes are redirected to the **AIP process** (Doc 06).
- **Security report** — *disabled as public issue*; routes to private advisory + Security Council per `SECURITY.md` (responsible disclosure).
- **Conformance failure** — failing vector id, role, implementation under test.

5. Pull Request Template

```

## what & why
## Linked AIP (required for protocol/API/schema changes)
## Backward compatibility (PATCH | MINOR | MAJOR — Doc 06 §5)
## Conformance (suite run output, must be green)
## Docs updated (which of Docs 01-15)
## Checklist
| | DCO sign-off
| | Matches OpenAPI (Doc 08) & JSON Schemas (Doc 10)
| | Code style (Doc 13)
| | Tests + conformance pass

```

6. Contribution Guide (summary)

1. For protocol/API/schema changes, **open an AIP first** (Doc 06) — code follows an Accepted AIP.
2. Small fixes (docs, examples, SDK bugs) go straight to PR with DCO sign-off.
3. CI must pass: `lint · test · openapi-validate · schema-validate · conformance`.
4. Keep invariants consistent (pricing, fees, 12 networks, receipt TTL) — any change must propagate to every affected doc/artifact (see README "Synchronization").
5. License: Apache-2.0. By contributing you agree to the DCO.

7. Release & Publishing

- **SemVer** tags trigger the release workflow: build, test, conformance, publish to the language registry (npm/PyPI/Go/crates/Maven/Packagist/NuGet), generate changelog, and deploy a docs preview.
- Protocol releases (aifp) drive a coordinated SDK bump; SDKs declare the protocol version(s) they conform to.
- kid rotation for receipt signing keys is a server-side release event documented in the changelog; old kid s stay resolvable for receipt TTLs.